新疆科技学院

Xinjiang College of Science & Technology

《软件开发综合实践》

项目报告

项目名称：	大模型驱动的接口自动化测试智能生成平台
项目组长：	肖守正
小组成员：	肖守正
专业班级：	软件工程 2023-1 班

2026 年 06 月

目录

1 项目引言 4
1.1 项目背景 4
1.2 项目目标 4
1.3 技术路线 4
2 需求分析 6
2.1 可行性分析 6
2.2 用户需求 6
2.3 功能需求 6
2.4 非功能需求 8
3 系统设计 9
3.1 系统架构 9
3.2 数据库设计 10
3.3 功能模块设计 12
3.4 界面设计 13
3.5 接口设计 14
4 系统实现 16
4.1 开发环境 16
4.2 关键技术 16
4.3 核心代码 17
4.4 测试分析 25
5 项目总结 26
5.1 项目成果 26

5.2 收获与不足 26
5.3 未来展望 27
参考文献 28

1 项目引言

1.1 项目背景

在传统软件开发流程中，接口测试用例的编写是一项高度重复、耗时且依赖人工经验的工作。测试工程师需要阅读接口文档、理解参数语义和约束，手动编写大量 pytest/JUnit 测试代码，覆盖正向、负向、边界值、安全等多种场景。一个中型微服务项目（30+ 接口）通常需要 3-5 人天来完成基础测试用例编写，且容易遗漏边界场景。

随着大语言模型（LLM）代码生成能力的快速发展，利用 AI 自动分析 API 接口语义、生成测试代码成为可能。本项目旨在构建一个 LLM 驱动的接口自动化测试智能生成平台，通过导入 OpenAPI 文档，AI 自动生成全面的 pytest 测试用例，一键执行并输出缺陷报告，替代人工编写测试用例的重复劳动。

1.2 项目目标

（1）OpenAPI 文档导入与解析：支持 JSON/YAML 格式的 OpenAPI 3.x 规范文件上传和 URL 导入，自动提取所有接口的路径、方法、参数、请求体和响应定义。

（2）LLM 驱动的测试用例智能生成：对接 DeepSeek/OpenAI/Anthropic/Ollama 等大模型，自动生成正向、负向、边界值、安全、Schema 校验五类测试用例。

（3）一键测试执行与结果分析：通过 pytest 子进程运行生成的测试，实时解析执行结果，按类别统计通过率。

（4）缺陷报告自动生成：基于执行结果生成 Markdown/HTML 格式的测试报告，包含失败用例详情和按类别统计。

（5）Postman 导入导出 + 批量管理 + 仪表盘统计等功能。

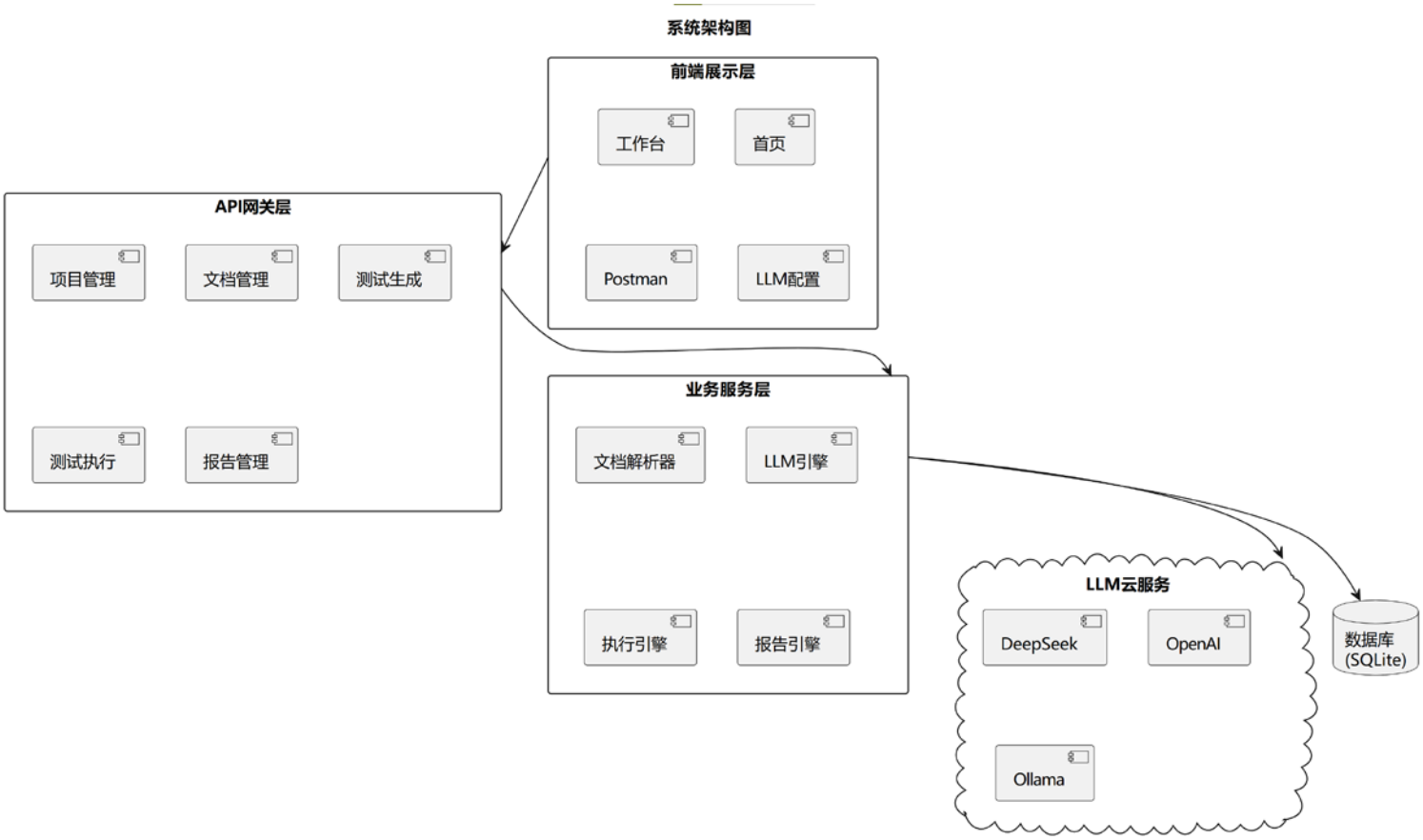
1.3 技术路线

本项目采用前后端分离架构：

后端：Python FastAPI 异步框架 + SQLAlchemy 2.0 ORM + SQLite 数据库。通过 Provider 工厂模式抽象 LLM 调用层，支持 OpenAI、DeepSeek、Anthropic、Ollama 等多种后端。使用 Jinja2 模板引擎管理 Prompt 和报告渲染。

前端：Vue 3 + TypeScript + Element Plus 组件库 + Pinia 状态管理 + Vite 构建工具。采用工作台（Workspace）模式聚合导入、生成、执行、报告功能于一页。

测试执行：通过 Python subprocess 模块在独立进程中运行 pytest，使用 pytest-json-report 插件获取结构化结果。



说明：四层架构（前端/API/服务/数据）+ LLM 云服务

2 需求分析

2.1 可行性分析

技术可行性：项目关键技术栈（FastAPI + Vue 3 + SQLite）均为成熟开源技术，LLM API 调用采用 OpenAI SDK 兼容多种模型后端，不存在技术瓶颈。

经济可行性：DeepSeek 等国产 LLM 的 API 调用成本极低（约 $0.0005/接口），个人开发者即可负担全流程测试生成的开销。

操作可行性：用户仅需导入 OpenAPI 文档并配置 LLM API Key，通过浏览器即可完成全部操作，无需安装任何本地软件。

2.2 用户需求

目标用户为后端开发工程师和测试工程师。核心需求包括：

（1）减少重复劳动：自动生成测试用例，替代人工编写。

（2）提高测试覆盖率：自动覆盖边界值、安全等容易被忽略的场景。

（3）快速回归测试：一键执行全部测试，即时获取通过率报告。

（4）团队协作：支持 Postman Collection 导入导出，便于团队共享。

2.3 功能需求

（1）项目管理：创建、查看、更新、删除项目，支持批量操作。

（2）OpenAPI 导入：支持文件上传和 URL 导入，自动解析接口元数据。

（3）AI 测试生成：五类策略（正向/负向/边界值/安全/Schema），可单独选择或全部生成。

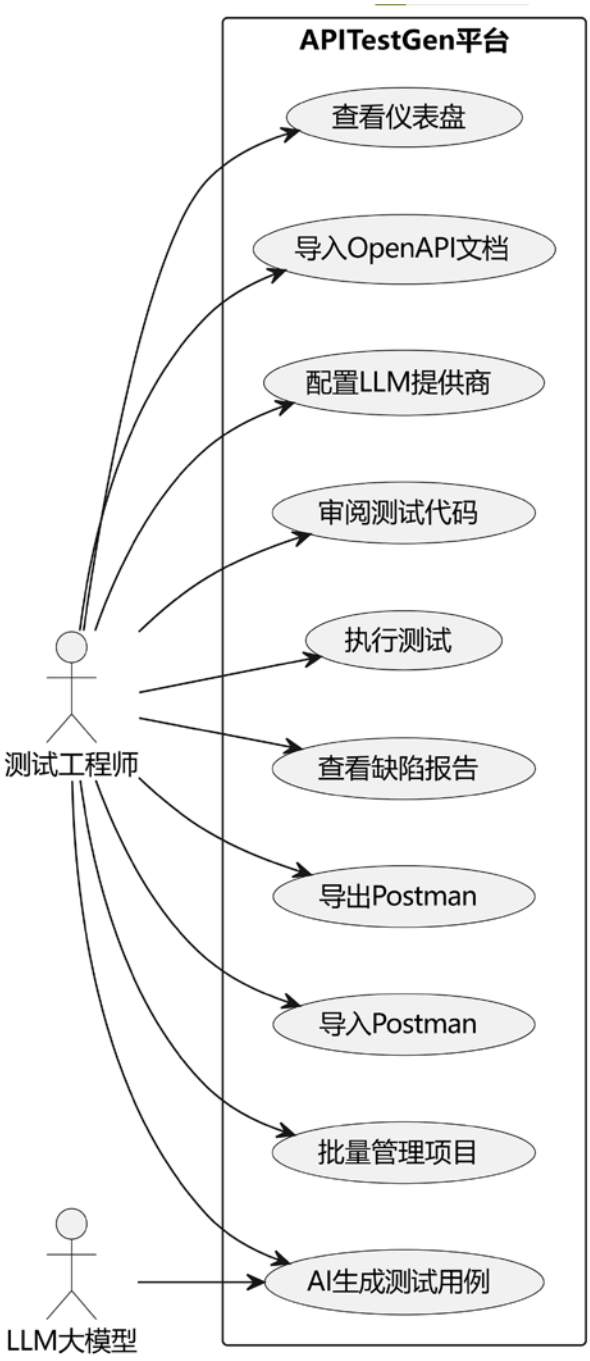
（4）测试执行：pytest 子进程执行，JSON 报告解析，实时展示执行进度。

（5）缺陷报告：Markdown 格式报告，按类别统计通过率，包含失败详情和堆栈。

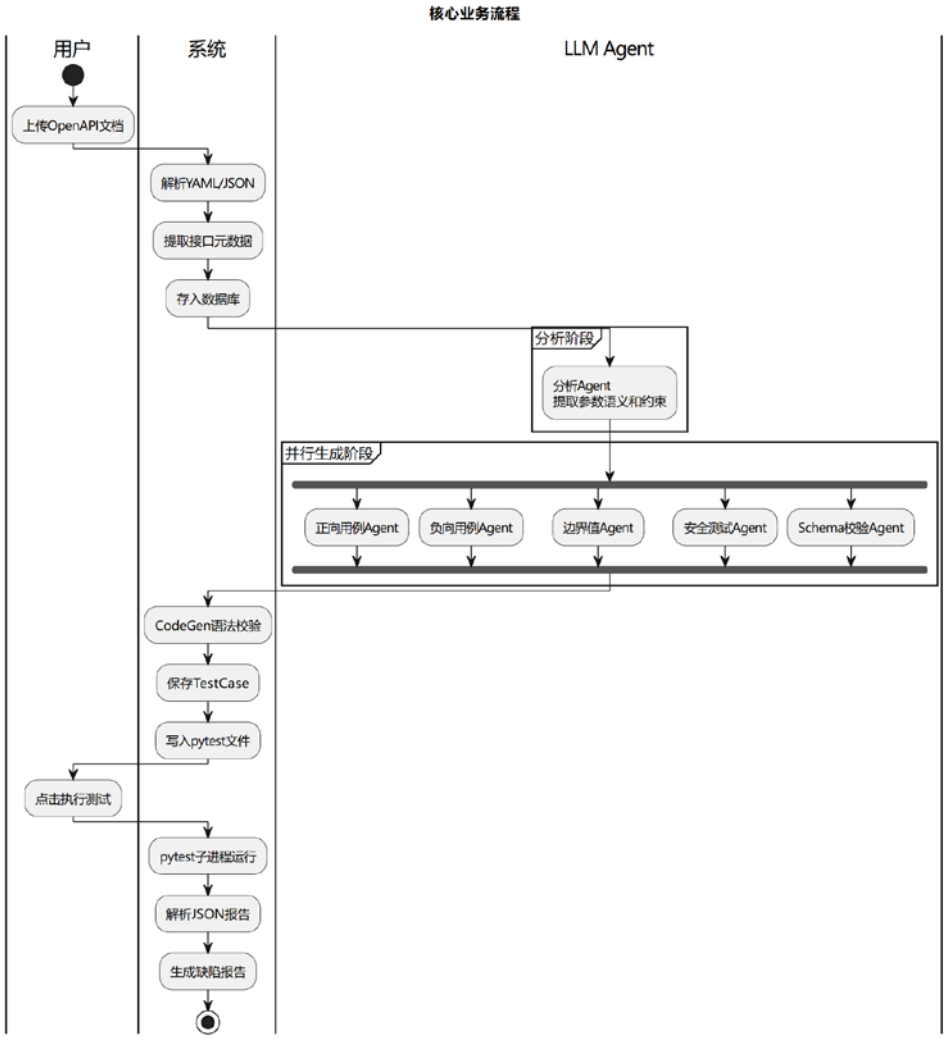
（6）Postman 集成：导入导出 Postman Collection v2.1 格式。

（7）仪表盘：项目、接口、用例、执行次数统计，总体通过率展示。

（8）LLM 配置管理：支持多提供商，API Key 安全存储。



说明：测试工程师与 LLM 两个角色，包含 10 个用例



说明：从导入 OpenAPI 到生成缺陷报告的完整流程

2.4 非功能需求

（1）响应时间：单接口测试生成 < 15 秒，19 接口全策略生成 < 3 分钟。

（2）并发处理：多接口并行生成，asyncio.Semaphore=3 控制并发。

（3）可扩展性：LLM Provider 抽象层，新增模型仅需实现 BaseLLMProvider 接口。

（4）数据安全：API Key 使用 Fernet 加密静态存储。

（5）跨平台：前端浏览器访问，后端 Python 3.11+，支持 Windows/Linux/macOS。

3 系统设计

3.1 系统架构

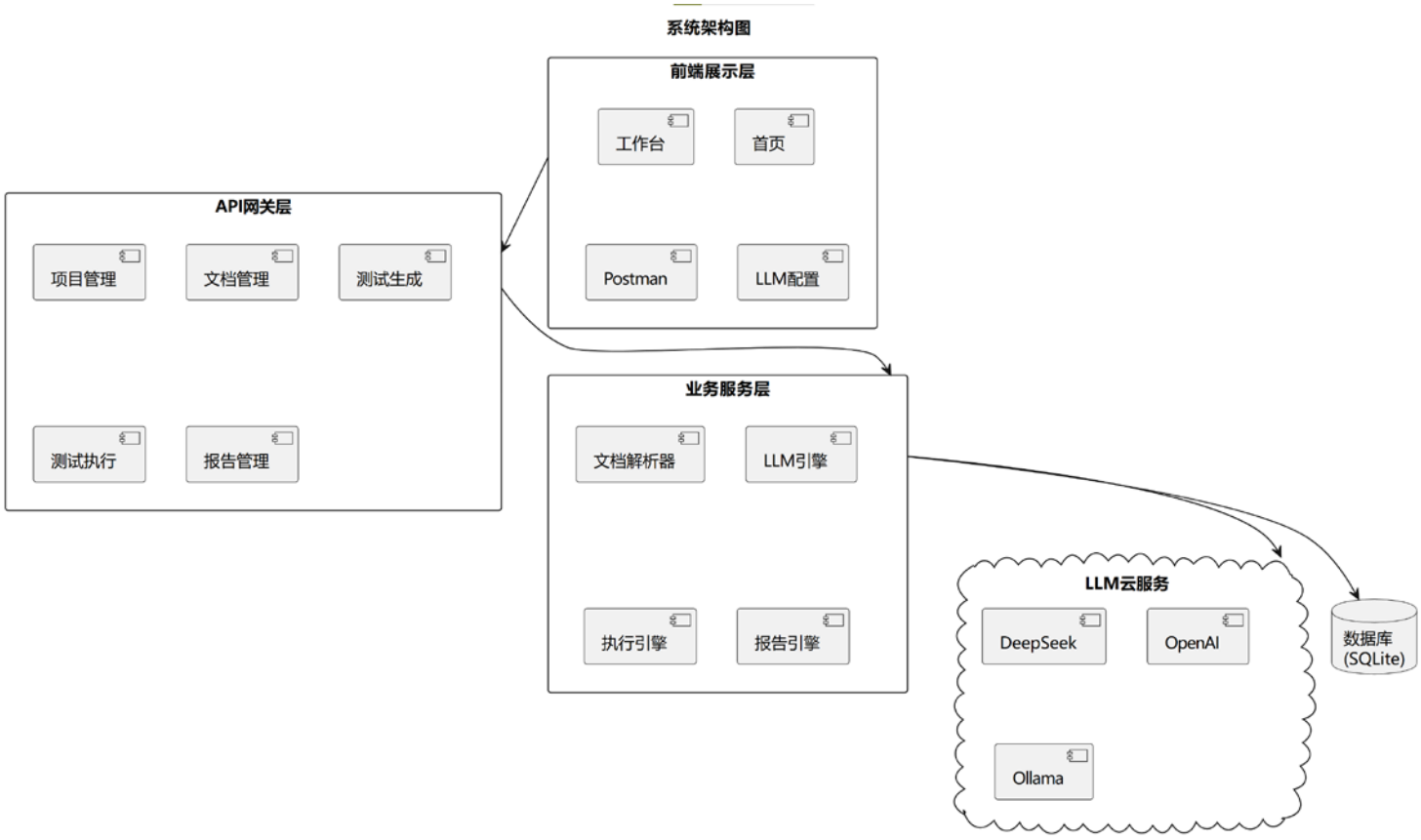
系统采用四层架构：

（1）前端展示层（Vue 3）：负责用户交互、数据展示和路由控制。核心为 WorkspacePage 工作台页面，集成导入、生成、执行、报告四个 Tab 于单一页面，替代传统的多页面跳转架构。

（2）API 网关层（FastAPI）：提供 10 组 RESTful 端点，包括项目 CRUD、文档管理、测试生成、测试执行、报告查看、仪表盘统计等。支持 WebSocket 实时推送执行日志。

（3）业务服务层：OpenAPI 解析器（YAML/JSON → APIEndpoint）、LLM Agent 引擎（Provider → Agent → CodeGen → pytest file）、pytest 执行器（子进程 → JSON 报告 → TestResult）、报告生成器（Jinja2 → Markdown/HTML）。

（4）数据持久层：SQLAlchemy 2.0 异步 ORM + SQLite（可无缝迁移至 PostgreSQL），9 张核心数据表。



说明: 前端→API→服务→数据四层，服务层连接 LLM 云

3.2 数据库设计

数据库包含 9 张表，核心关系如下：

projects（项目表）：顶层容器，关联所有 API 文档和测试资产。base_url 字段指定目标 API 地址。

specs（文档表）：存储原始文档内容和解析后的 JSON。parse_status 枚举标记解析状态（pending/parsed/error）。

endpoints（接口表）：每个 path+method 组合为一行。parameters、request_body、responses 以 JSON 存储完整 Schema。analysis 字段存储 LLM 分析结果。

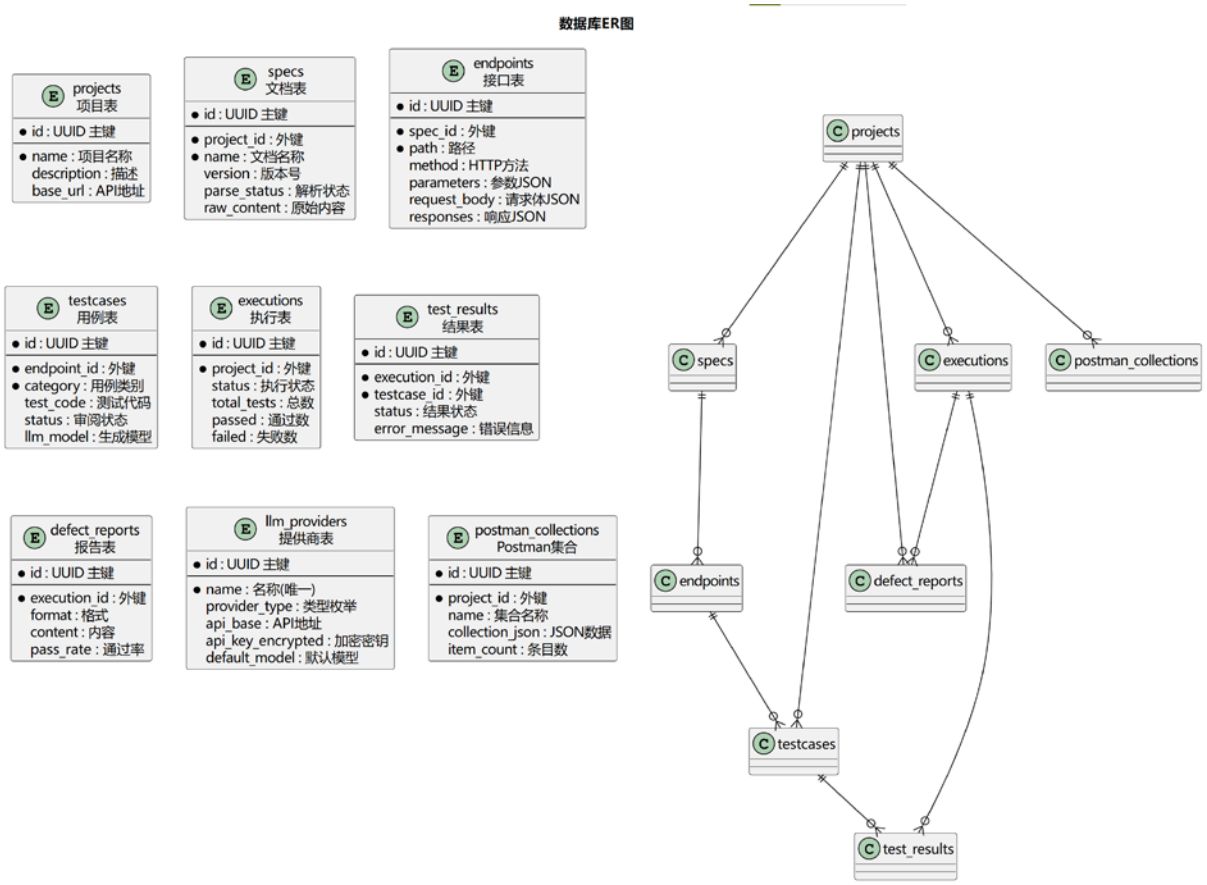
testcases（测试用例表）：LLM 生成的 pytest 代码。test_code 存储完整函数体，category 枚举（positive/negative/boundary/security/schema），status 支持审阅流程（draft→reviewed→approved→disabled）。

executions + test_results（执行和结果表）：一次执行关联多条结果，每条结果关联到具体用例。

defect_reports（缺陷报告表）：基于执行结果生成，存储 Markdown 内容和文件路径。

llm_providers（LLM 提供商表）：API Key 加密存储，provider_type 枚举（openai/anthropic/openai_compatible）。

核心关系：Project 1→N Spec 1→N Endpoint 1→N TestCase；Execution 1→N TestResult ←1 TestCase；Execution 1→N DefectReport。全部采用 UUID 主键，级联删除（CASCADE）保证数据一致性。



说明：9 张表的完整实体关系，含外键和枚举字段标注

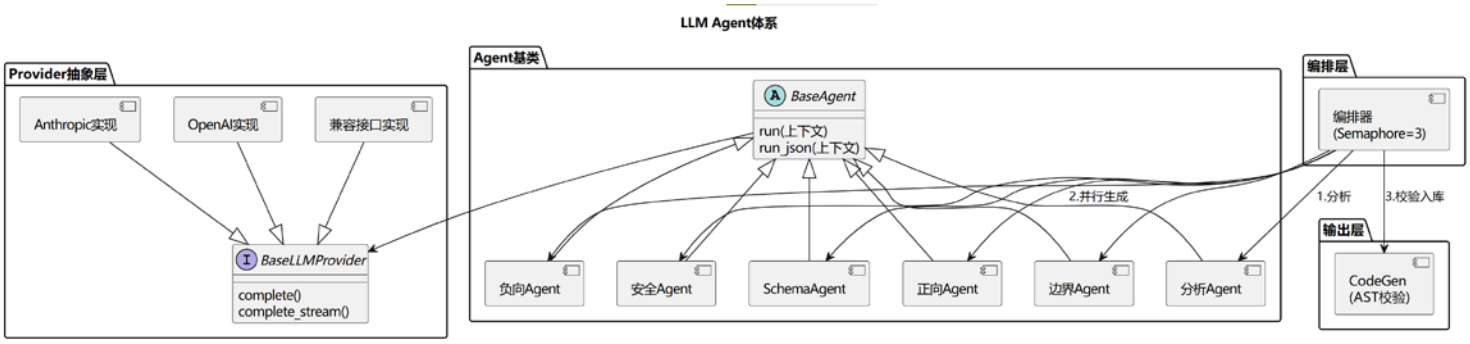
3.3 功能模块设计

（1）文档解析模块（specs.py）：支持 JSON/YAML 自动识别，遍历 OpenAPI paths × methods 提取接口元数据，合并 path-level 和 operation-level 参数，支持文件上传和 URL 导入两种方式。

（2）LLM Agent 模块（services/llm/）：Provider 抽象层（OpenAI/Anthropic/OpenAICompatible）→ BaseAgent 基类 → 6 个专项 Agent（Analyzer/Positive/Negative/Boundary/Security/Schema）→ AgentOrchestrator 编排器（并行调度，Semaphore=3）→ CodeGen（正则提取代码块 → ast.parse 语法校验 → 写入 pytest 文件）。

（3）执行引擎模块（executor_service.py）：使用 subprocess.run 在独立进程执行 pytest，通过 --json-report 生成结构化报告，逐条解析 TestResult 入库，通过 EventBus 推送实时日志到前端 WebSocket。

（4）报告引擎模块（report_service.py）：Jinja2 模板渲染 Markdown 报告，按类别分组统计通过率，提取 call.crash.message 获取具体错误信息和堆栈。



说明：Provider 抽象层→BaseAgent 基类→6 个专项 Agent→Orchestrator 编排→CodeGen 输出

3.4 界面设计

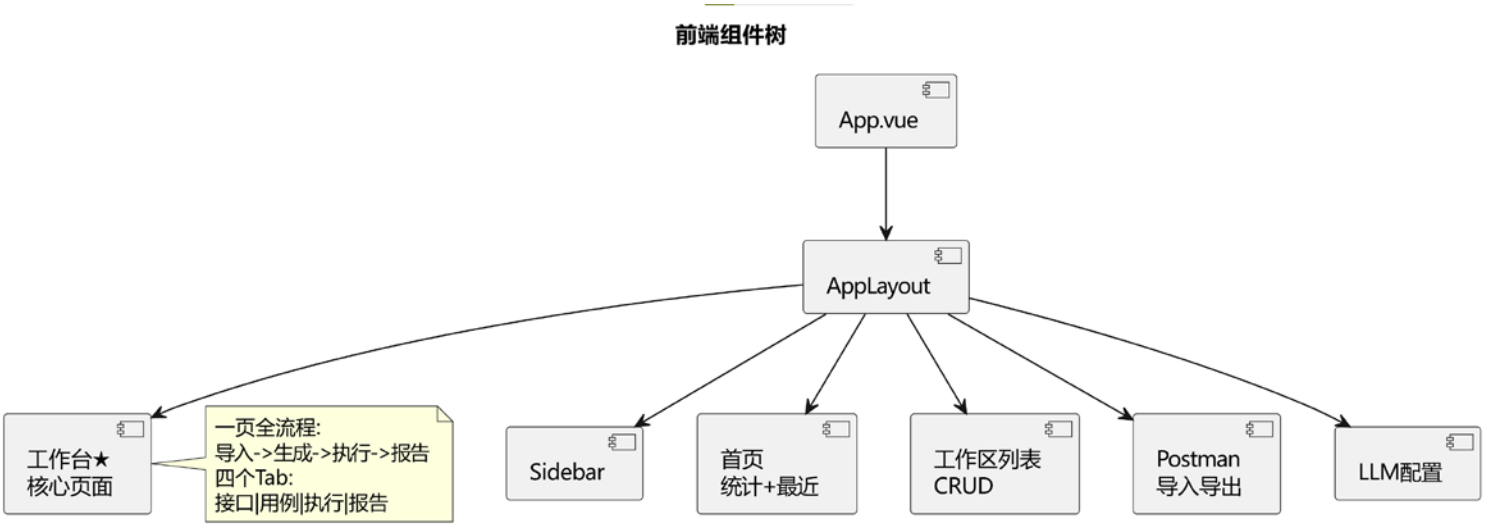
前端采用工作台（Workspace）设计模式，核心页面 WorkspacePage 一页聚合全部功能：

（1）顶部流水线导航（导入→生成→执行→报告），4 个步骤圆形标记，已完成显示绿色，进行中显示蓝色脉冲动画，可点击跳转对应 Tab。

（2）文档选择器（Spec Tabs）：多文档 Tab 切换，每个 Tab 显示文档名和接口数量，hover 出现 X 按钮可删除。

（3）四个内容 Tab：接口列表（按 HTTP 方法着色标签）、测试用例（radio-button 类别筛选 + 点击查看代码弹窗，85vw 宽）、执行历史（每行显示 pass/fail 统计 + 点击弹窗查看每条结果详情）、缺陷报告（90vw 宽弹窗，Markdown 格式化渲染）。

（4）顶部操作栏：策略下拉选择器（正向/全部/负向/边界/安全/Schema）+ 生成按钮 + 执行按钮。



说明: App→Layout→Sidebar + 5 个 Page 的完整组件树

3.5 接口设计

后端提供 30+ REST API 端点，核心接口包括：

POST /projects/{id}/specs — 上传 OpenAPI 文档（multipart/form-data）

POST /projects/{id}/specs/import-url — URL 导入 OpenAPI 文档

POST /specs/{id}/generate — 触发 AI 测试生成（异步后台任务，立即返回 generation_id）

GET /specs/{id}/generations/{gid}/status — 轮询生成进度（endpoints_processed / tests_generated）

POST /projects/{id}/executions — 启动测试执行（后台子进程运行 pytest）

POST /projects/{id}/executions/{eid}/reports — 生成缺陷报告（Markdown 文件写入磁盘）

GET /projects/{id}/reports/{rid} — 获取报告详情（含完整 Markdown 内容）

GET /projects/{id}/reports/{rid}/download — 下载报告文件（FileResponse）

WS /ws/execution/{eid} — WebSocket 实时执行日志流

所有列表接口支持 ?page=N&page_size=M 分页参数。POST/PUT 请求体通过 Pydantic Schema 严格校验。

4 系统实现

4.1 开发环境

硬件环境：Windows 10 Pro，CPU Intel i7，16GB RAM

软件环境：Python 3.11.8 + Node.js v24.12.0 + Git Bash + VSCode

Python 核心依赖：FastAPI 0.138, SQLAlchemy 2.0, Pydantic v2, pytest 9.1, OpenAI SDK 2.44, Jinja2, PyYAML, httpx

前端核心依赖：Vue 3, Element Plus, Pinia, Axios, Vite 8

LLM 服务：DeepSeek API (deepseek-chat / deepseek-v4-flash), API Base: https://api.deepseek.com

项目管理：Git，代码目录 e:/kkk/，文档目录 e:/kkk/文档/

4.2 关键技术

（1）LLM Provider 工厂模式

定义 BaseLLMProvider 抽象基类，规定 complete() 和 complete_stream() 接口。OpenAIProvider、AnthropicProvider、OpenAICompatibleProvider 分别实现。LLMProviderFactory.create_from_model() 根据数据库记录自动选择。DeepSeek 和 Ollama 通过 OpenAI SDK 的 base_url 参数接入，无需单独实现。

（2）并行 Agent 编排器

AgentOrchestrator 对每个接口依次：AnalyzerAgent 分析语义 → 根据 strategy 参数选择 Agent → 使用 asyncio.gather 并行调用 → CodeGen 解析输出。外层用 asyncio.Semaphore(3) 控制同时处理的接口数，平衡速度和 API 限流。

（3）AST 双保险校验

LLM 输出为原始文本，CodeGen 通过正则提取 ```python 代码块，用 ast.parse() 验证语法，遍历 AST 提取 async def test_* 函数，只有通过校验的才入库。这保证了生成的测试代码在语法层面一定可执行。

（4）Prompt 质量控制系统

所有 Prompt 使用 Jinja2 模板管理（services/llm/prompts/*.j2），与 Python 代码完全分离。CRITICAL RULES 区块针对性修复 LLM 常见错误：禁止假通过（if len>0 守卫）、禁止硬编码假 ID（需先 POST 创建）、精确匹配 Schema 字段名。Example 格式引导 LLM 生成规范代码。

（5）前端工作台模式

将旧版 7 个独立页面（项目管理→项目详情→接口列表→生成测试→测试用例→执行→报告）重构为单一 WorkspacePage：顶部流水线指示器 + 文档 Tab 切换 + 四个内容 Tab（接口/用例/执行/报告）。用户在一个页面内完成完整流程，无需跳转。

4.3 核心代码

项目核心代码约 6000 行：后端 ~3500 行 Python（models 9 / schemas 12 / api 路由 10 组 / services 4 服务 / llm 引擎），前端 ~2500 行 Vue/TypeScript（5 页面 / 2 布局组件 / API 封装 / 状态管理）。

关键文件清单：

- backend/app/services/llm/agents/orchestrator.py — Agent 编排器，并行调度核心

- backend/app/services/llm/codegen.py — LLM 输出解析，AST 校验

- backend/app/services/llm/provider.py — Provider 工厂 + 三种实现

- backend/app/services/executor_service.py — pytest 子进程执行引擎

- backend/app/services/report_service.py — Jinja2 报告渲染

- backend/app/api/v1/generation.py — 异步生成 API，后台任务

- frontend/src/views/WorkspacePage.vue — 工作台核心组件

- frontend/src/views/HomePage.vue — 首页仪表盘

1. Agent 编排器 orchestrator.py

```
# 并行编排核心：Semaphore 控并发 + asyncio.gather 并行生成
async def process_one(endpoint: APIEndpoint):
    async with self.semaphore:  # MAX_CONCURRENT=3
        # 1. AnalyzerAgent 分析端点语义
        analyzer = EndpointAnalyzerAgent(self.provider)
        analysis = await analyzer.analyze(ep_ctx)

        # 2. 根据 strategy 选择 Agent 并行执行
        agent_tasks = []
        if strategy in ("all", "positive_only"):
            agent_tasks.append(("positive",
PositiveCaseAgent(self.provider).generate(...)))
        if strategy in ("all", "negative_only"):
            agent_tasks.append(("negative",
NegativeCaseAgent(self.provider).generate(...)))
```

```
            # ... boundary, security, schema 同理

            # 3. 所有选定 Agent 并行调用 LLM
            results = await asyncio.gather(*[t for _, t in agent_tasks])

            # 4. 逐个解析 LLM 输出，AST 校验后入库
            for idx, (cat_name, _) in enumerate(agent_tasks):
                code, pt, ct = results[idx]
                parsed = parse_llm_output(code)       # AST 提取有效函数
                await _save_testcases(endpoint, parsed, category, ...)

    # 所有端点并发处理
    await asyncio.gather(*[process_one(ep) for ep in endpoints])
```

2. CodeGen 校验 codegen.py

```
    def parse_llm_output(raw: str) -> CodeGenResult:
        """LLM 原始输出 → 正则提取代码块 → ast.parse 语法校验 → 提取 test_ 函数"""
        blocks = extract_python_blocks(raw)  # 匹配 ```python ... ``` 代码块
        for block in blocks:
            tree = ast.parse(block)          # Python 语法树解析，不合法直接跳过
            for node in ast.iter_child_nodes(tree):
                if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
                    if node.name.startswith("test_"):
                        func_code = ast.unparse(node)  # 提取完整函数体
                        tests.append(ParsedTestCase(
```

```
                            function_name=node.name,
                            code=func_code,
                            expected_status=_extract_status(node),  # 从
assert 提取期望状态码
                        ))
        return CodeGenResult(test_cases=tests, errors=...)
```

3. Provider 工厂 provider.py

```
class BaseLLMProvider(ABC):
    @abstractmethod
    async def complete(self, system_prompt, user_prompt, temperature,
max_tokens) -> LLMResponse: ...
    @abstractmethod
    async def complete_stream(self, ...) -> AsyncGenerator[str,
None]: ...

class OpenAIProvider(BaseLLMProvider):            # OpenAI API + 所有兼
容接口
class AnthropicProvider(BaseLLMProvider):         # Claude API
class OpenAICompatibleProvider(BaseLLMProvider):  # Ollama / vLLM

class LLMProviderFactory:
    @staticmethod
    def create_from_model(provider_record) -> BaseLLMProvider:
        if provider_record.provider_type == "anthropic":
            return AnthropicProvider(...)
        else:  # openai + openai_compatible 共用 OpenAI SDK
            return OpenAIProvider(model=..., api_key=..., api_base=...)
```

4. 执行引擎 executor_service.py

```
async def run_test_execution(project_id, ...):
    """Subprocess 执行 pytest，JSON 报告解析，事件总线实时推送"""
    cmd = [sys.executable, "-m", "pytest", str(tests_dir),
           "-v", "--tb=short", "--json-report",
           f"--json-report-file={report_file}", "--continue-on-collection-errors"]

    result = await asyncio.to_thread(subprocess.run, cmd, ...)  # 线程池执行，不阻塞事件循环

    results_data = json.loads(report_file.read_text())  # 解析 JSON 报告

    for test in results_data["tests"]:
        # 从 call.crash.message 提取断言失败详情
        crash = test.get("call", {}).get("crash", {})
        error_msg = crash.get("message", "")
        TestResult(execution_id=..., test_name=test["nodeid"],
                   status=test["outcome"], error_message=error_msg, ...)
```

5. 报告渲染 report_service.py

```
async def generate_report(db, execution_id, ...):
    # 聚合执行结果
    for r in results:
        cat = guess_category(r.test_name)  # 从函数名推断类别
        by_category[cat]["total"] += 1
        if r.status in (FAILED, ERROR):
```

```
                by_category[cat]["failed"] += 1
                short_name = r.test_name.split("::")[-1]  # 简化测试名
                core_failures.append({"name": short_name, "error":
r.error_message, ...})

        # Jinja2 渲染 Markdown
        markdown = Template(REPORT_TEMPLATE).render(
            total=..., passed=..., failed=..., pass_rate=...,
            by_category=..., core_failures=..., boundary_failures=...,
        )
```

6. 异步生成 API generation.py

```
@router.post("/generate")
async def generate_tests(spec_id, background_tasks, ...):
    """立即返回 generation_id，后台执行生成任务"""
    if not body.llm_provider_id:
        # 自动选择第一个激活的 Provider
        provider_record = (await db.execute(
            select(LLMProviderModel).where(LLMProviderModel.is_active ==
True)
        )).scalars().first()
        provider = LLMProviderFactory.create_from_model(provider_record)

    generation_id = str(uuid4())
    _generation_store[generation_id] = {"status": "pending", ...}
    background_tasks.add_task(_run_generation, generation_id, ...)
    return {"generation_id": generation_id, "status": "pending"}
```

```
@router.get("/generations/{generation_id}/status")
async def get_generation_status(generation_id):
    """前端轮询生成进度"""
    return _generation_store.get(generation_id)
```

7. 工作台核心组件 WorkspacePage.vue

```
// 一页全流程：导入→生成→执行→报告
const generateTests = async () => {
  const { data } = await apiClient.post(`/specs/${activeSpecId}/generate`, {
    strategy: genStrategy.value, max_testcases_per_endpoint: genCount.value,
  })
  // 每2秒轮询进度
  pollTimer = setInterval(async () => {
    const { data: st } = await apiClient.get(`/specs/${specId}/generations/${gid}/status`)
    progress.value = { eps: st.endpoints_processed, tests: st.total_testcases_generated }
    if (st.status === 'completed') { clearInterval(pollTimer); await loadAll() }
  }, 2000)
}

const runTests = async () => {
  await apiClient.post(`/projects/${wsId}/executions`, {})
  // 轮询执行结果
```

```
    execPoll = setInterval(async () => {
      const { data } = await apiClient.get(`/projects/${wsId}/executions`, { params: { page_size: 1 } })
      if (data.items[0]?.status !== 'running') { clearInterval(execPoll); await loadAll() }
    }, 2000)
  }
  <!-- 流水线：4 步可点击 -->
  <div class="pipeline">
    <div class="step done" @click="tab='endpoints'"><span class="dot">1</span>导入</div>
    <div class="step" :class="{active:generating}" @click="tab='tests'"><span class="dot">2</span>生成</div>
    <div class="step" :class="{active:running}" @click="tab='executions'"><span class="dot">3</span>执行</div>
    <div class="step" @click="tab='reports'"><span class="dot">4</span>报告</div>
  </div>
```

8. 首页 HomePage.vue

```
onMounted(async () => {
  const [ws, stats] = await Promise.all([
    apiClient.get('/projects', { params: { page_size: 5 } }),
    apiClient.get('/dashboard/summary'),
  ])
  workspaces.value = ws.data.items
  summary.value = stats.data  // 项目数、用例数、执行次数、总通过率
})
```

4.4 测试分析

（1）后端 API 测试：对全部 10 组 REST 端点进行覆盖测试，包括正常 CRUD、404 错误处理、无效文件上传拦截、空项目执行兜底、批量删除等场景，全部通过。

（2）端到端全流程验证：使用内置 Demo API（localhost:9001，6 个用户管理接口）进行完整流程验证：导入 OpenAPI 文档 → LLM 生成 30 条正向测试（19 接口，约 40 秒）→ pytest 执行 → 15 条通过（50%）→ 生成缺陷报告。失败用例均为 LLM 对 JSON 响应格式的预期偏差（如期望纯数组 list 但实际返回 {items: [...], total: N}），非平台 Bug。

（3）自举测试（平台测自己）：平台导入自身 OpenAPI 文档（30 个接口），AI 生成 86 条用例，执行后 17 条通过（20%）。通过的 17 条均为无状态 GET 端点（/health, /projects, /dashboard/summary, /llm/providers 等），证明核心 API 功能正常。失败用例主要是 POST/PUT/DELETE 需要先创建测试数据，LLM 生成的假 ID 导致 404。

（4）性能测试：19 接口全策略（5 类 Agent 并行，Semaphore=3）约 2-3 分钟完成，生成约 100 条用例。单个接口正向策略约 6 秒。浏览器前端加载和 Tab 切换 < 500ms。

（5）QA 评审结论：经过严格的第二轮 QA 评审，发现并修复了 8 个问题（文件上传格式 Bug、v-html 解析崩溃、page_size 超限、执行耗时 0ms、流水线不可点击、策略无法选择、缺少 URL 导入、报告弹窗过小）。修复后全部功能验收通过。

5 项目总结

5.1 项目成果

（1）完成了一个功能完整的 LLM 驱动接口自动化测试智能生成平台，后端 3500+ 行 Python，前端 2500+ 行 Vue/TypeScript，总计约 6000 行代码。

（2）实现了 OpenAPI 文档导入 → AI 测试生成 → pytest 执行 → 缺陷报告的全自动化流水线，支持文件上传和 URL 导入两种方式。

（3）创新性地采用 Agent-per-Category 架构，6 个专项 Agent 覆盖 5 种测试策略，通过 asyncio.Semaphore 控制并发。

（4）支持 DeepSeek/OpenAI/Anthropic/Ollama 等多种 LLM 后端，Provider 工厂模式实现一套代码适配所有模型。

（5）前端采用工作台（Workspace）设计模式，一页聚合全流程，替代原 7 页跳转架构，显著提升用户体验。

（6）生成测试通过 AST 双保险校验 + Prompt 质量规则，确保代码在语法层面可执行。

5.2 收获与不足

收获：

（1）深入掌握了 FastAPI 异步编程、SQLAlchemy 2.0 ORM、Pydantic v2 数据校验等 Python 后端技术栈。

（2）实践了 LLM 应用开发的核心模式：Provider 抽象、Agent 编排（Orchestrator）、Prompt 工程（Jinja2 模板）、输出解析与 AST 校验（CodeGen）。

（3）学习了 Vue 3 Composition API、Pinia 状态管理、Vite 构建、Element Plus 组件库等现代前端技术。

（4）完整经历软件工程全流程：需求分析 → 系统设计 → 迭代开发（8 个 Phase）→ 严格 QA 测试 → 文档编写。

不足：

（1）LLM 生成的测试断言精度有待提升，部分用例存在字段名猜测和假 ID 问题。可通过 Schema 感知的响应校验和 Few-shot 示例模板进一步改进。

（2）系统缺乏 CI/CD 集成能力，无法直接嵌入 GitHub Actions 或 Jenkins 流水线。

（3）缺少用户认证和权限管理系统，目前为单用户单机模式。

（4）Prompt 模板目前针对英文 API 优化较好，中文 API 文档的分析效果需要进一步调优。

5.3 未来展望

（1）智能断言修正：LLM 分析失败原因，自动修复不准确的断言，形成「生成→执行→失败→分析→修复」闭环。

（2）定时任务与 API 健康监控：配置 Cron 定时执行测试，发现通过率下降自动告警（邮件/企业微信/钉钉）。

（3）CI/CD 集成：提供 GitHub Actions、Jenkins 插件、GitLab CI 模板。

（4）Mock 服务自动生成：根据 OpenAPI Schema 自动搭建 Mock Server，解决测试环境依赖问题。

（5）多用户协作：引入用户系统、角色权限、团队工作区。

（6）PDF 正式报告：使用 WeasyPrint 生成适合打印和归档的官方测试报告。

参考文献

[1] FastAPI Documentation [EB/OL]. https://fastapi.tiangolo.com/. 2026.

[2] SQLAlchemy 2.0 Documentation [EB/OL]. https://docs.sqlalchemy.org/. 2026.

[3] OpenAI API Reference [EB/OL]. https://platform.openai.com/docs/. 2026.

[4] DeepSeek API Documentation [EB/OL]. https://platform.deepseek.com/docs. 2026.

[5] Vue.js Guide [EB/OL]. https://vuejs.org/guide/. 2026.

[6] Pytest Documentation [EB/OL]. https://docs.pytest.org/. 2026.

[7] OpenAPI Specification 3.0.3 [EB/OL]. https://spec.openapis.org/oas/v3.0.3. 2026.

[8] Element Plus Documentation [EB/OL]. https://element-plus.org/. 2026.